

CS3160 Lab 3

Measuring a Machine, and Building Its Memory

What you will do today

There are two parts for this lab.

In the first part, you will *measure*. We will give you a working simulator and a handful of programs. You will execute the program on the simulated processor, read the numbers that come out, and work out what they mean — how many instructions a program executed, how many cycles that took, and why the ratio is not 1.

In the second part (and during the rest of the semester), you will *build* your own processor simulator step-by-step. We will focus on the memory interface (instruction and data memory) of this simulated processor. The simulated instruction memory is a flat array of bytes that programs can be loaded into and read back out of. In today's lab, we will focus only on loading a program into the main memory and showing you the bytes it is made of. We will not worry about what those bytes mean (are they instructions or data; if they are indeed instructions, is it a legal instruction; if it is a legal instruction, what type of instruction is it, etc) in today's lab.

► *As mentioned earlier, we will be using C++ to build the simulator. Run `make test` often to guide you through today's lab.*

1 Part 1: measuring the reference simulator

There are five programs given in `tests/images/`. A `reference-sim` has already been built and placed in `/opt/cs3160/`. Make sure `/opt/cs3160` is added to the `PATH` of your shell.

Run one:

```
reference-sim --proc=single tests/images/1-even.r5ob
```

The simulator writes `stats.json` and `sim.log` into the current directory, and prints a summary. The numbers you want are in the summary and in `stats.json`:

<code>instructions</code>	how many instructions actually retired
<code>cycles</code>	how long the machine took
<code>cpi</code>	the ratio of the two

Exercise 1.1

Run all five programs and record `instructions`, `cycles` and `cpi`. You will find every CPI is exactly 1.0000.

`-proc=single` with no configuration file simulates a single-cycle processor attached to an ideal memory — one that sends data instantaneously. Here, each and every instruction takes exactly one cycle.

Exercise 1.2

Now make the memory realistic. Create a file called `slow.ini`:

```
[RAM]
latency = 100
```

and run again with `-config=slow.ini`. Record the same three numbers.

The instruction count should not have changed — indicates that the program did the same amount of work. The cycle count has changed. Understand where did the extra cycles come from: how many memory accesses were there, and what did each cost? *The RAM accesses line in the summary should help you.*

Exercise 1.3

Put a cache in front of it. Add to `slow.ini`:

```
[I1_Cache]
valid      = true
latency    = 2
size       = 1024
block_size = 64
assoc      = 1
mapping    = direct
replacement = FIFO
write_policy = write-back
write_miss = allocate
```

Run again and record the hit rate and the cycle count. Then answer, in `answers.txt`, in a sentence each:

1. Why does a 1 KiB cache help a program whose instructions do not fit in 1 KiB?
2. Which program benefits least, and what is different about it?

► *Save these numbers (after the lab). We will revisit these numbers in future when your own simulator is ready.*

2 Part 2: the memory

Open `src/ram.cpp`. Three functions are missing, marked `TODO(week 3)`, and one more is missing from `src/loader.cpp`.

2.1 ram_read and ram_write

Memory is a flat array of `ram.bytes` bytes, and an address is an index into it (offset by `ram.base`). Reading four bytes at an address means collecting four entries (bytes) into one 32-bit word.

RISC-V is **little-endian**: the byte at the *lowest* address is the *least significant* one. So the four bytes 93 00 00 00 at address 0x80000000 correspond to the word 0x00000093 (and not 0x93000000).

► *Every value that comes out may be byte-reversed. If this happens, then nothing will crash, but the processor may interpret the word as an illegal instruction. **make test** should help you catch this bug.*

A read narrower than a word is *zero-extended* into the returned `u32`. Whether the sign-bit should be filled in (instead of zeros) depends on whether the instruction was `lb` or `lbu`.

2.2 ram_dump

Print the words in a range, one per line:

```
0x80000000: 00000093
0x80000004: 00000113
```

Note the output format: address, a colon and a space, then eight hex digits with leading zeros. Ensure that the lines are rounded to a word boundary so the columns line up (irrespective of whether one byte was request).

The tests compare this character for character and hence match the format exactly: lower-case hex, "0x%08x: %08x\n".

2.3 loader_load

The input executable file (`.r5ob` file) is not an ELF. It is exactly the bytes that belong in memory (in the same order, with no prefix or suffix). In a sense, it is a *dump* of the memory regions just before the program starts executing on the machine. So, to simulate loading a program into the memory is just opening the file, reading it, and putting the byte *n* at `load_addr + n`. The function expects you to return the count of bytes copied by it. This return value is used only for debugging our simulator.

Use the function `ram_write_byte()` rather than writing into `ram.bytes` directly, so that an image too big for the configured RAM is caught and reported here instead of failing silently by writing beyond the end of the array. A file that will not open should call `fatal()` with the path in the message.

3 Checking your work

```
make          # build build/sim
make test     # the tests: build, then unit, then programs
make unit     # just the unit tests, quieter while you work on one
               function
```

`make test` is the same checker used to grade your work.

There are three tiers of tests:

1. **build** — it compiles the code (catches syntax errors). Does not run anything.
2. **unit** — tests each function on its own. A failure names the function and the case.
3. **programs** — your simulator loads each of the five programs and dumps the first 512 bytes. This compare the output against reference.

Fix the failing unit tests before looking at programs.

4 Submission steps

Your folder is collected at the end of the session and **only what is in it at that moment is graded**. So do not forget to save your work. Also ensure the following before you leave:

1. `details.txt` — your name, roll number and machine number. If this is blank, your submission cannot be matched to you.
2. `answers.txt` — the tables from Part 1 and the two sentences from Exercise 1.3.
3. `src/ram.cpp` and `src/loader.cpp` — your code.

You may keep working afterwards for your own practice but nothing you do after the session counts (as before).